Group: Sara Martinez Rendon (1000424179) | Heider Zapata (1000009586) | Yeison Londoño (1000009479)
Master's Degree in Data Sciences | EAFIT University
Course: Applied Statistics
Date: 27/03/2026

Examen 3: Modelo de regresión

---

0. Configuración del entorno y carga de datos externa

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Herramientas para Inferencia y Diagnósticos (Pregunta 1)
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, het_white
from scipy import stats

# Herramientas para Modelado Predictivo y LASSO (Pregunta 2)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures, LabelEncoder
from sklearn.linear_model import Lasso, LassoCV, LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from statsmodels.stats.stattools import durbin_watson
import warnings


# Configuración de visualización
%matplotlib inline
sns.set(style="whitegrid")

# Definición de enlaces de descarga directa desde Google Drive
url_consumo = "https://drive.google.com/uc?id=1R0Uj-ws7OsCnekJ_IHB3OErVYRE-0vlM"
url_mantenimiento = "https://drive.google.com/uc?id=1VPTb7YgV2P4vk8m3BH7pJEmlDcPwi_ly"

# Carga de los datasets
try:
    df_consumo = pd.read_csv(url_consumo)
    df_mantenimiento = pd.read_csv(url_mantenimiento)
    print("Datasets cargados exitosamente.")
    print(f"Dimensiones Consumo Energía: {df_consumo.shape}")
    print(f"Dimensiones Flota Mantenimiento: {df_mantenimiento.shape}")
except Exception as e:
    print(f"Error al cargar los datos: {e}")
```

```
Datasets cargados exitosamente.
Dimensiones Consumo Energía: (800, 7)
Dimensiones Flota Mantenimiento: (500, 6)
```

---

1. Inferencia y Diagnóstico Econométrico

1.1. Análisis preliminar de Correlación y Multicolinealidad

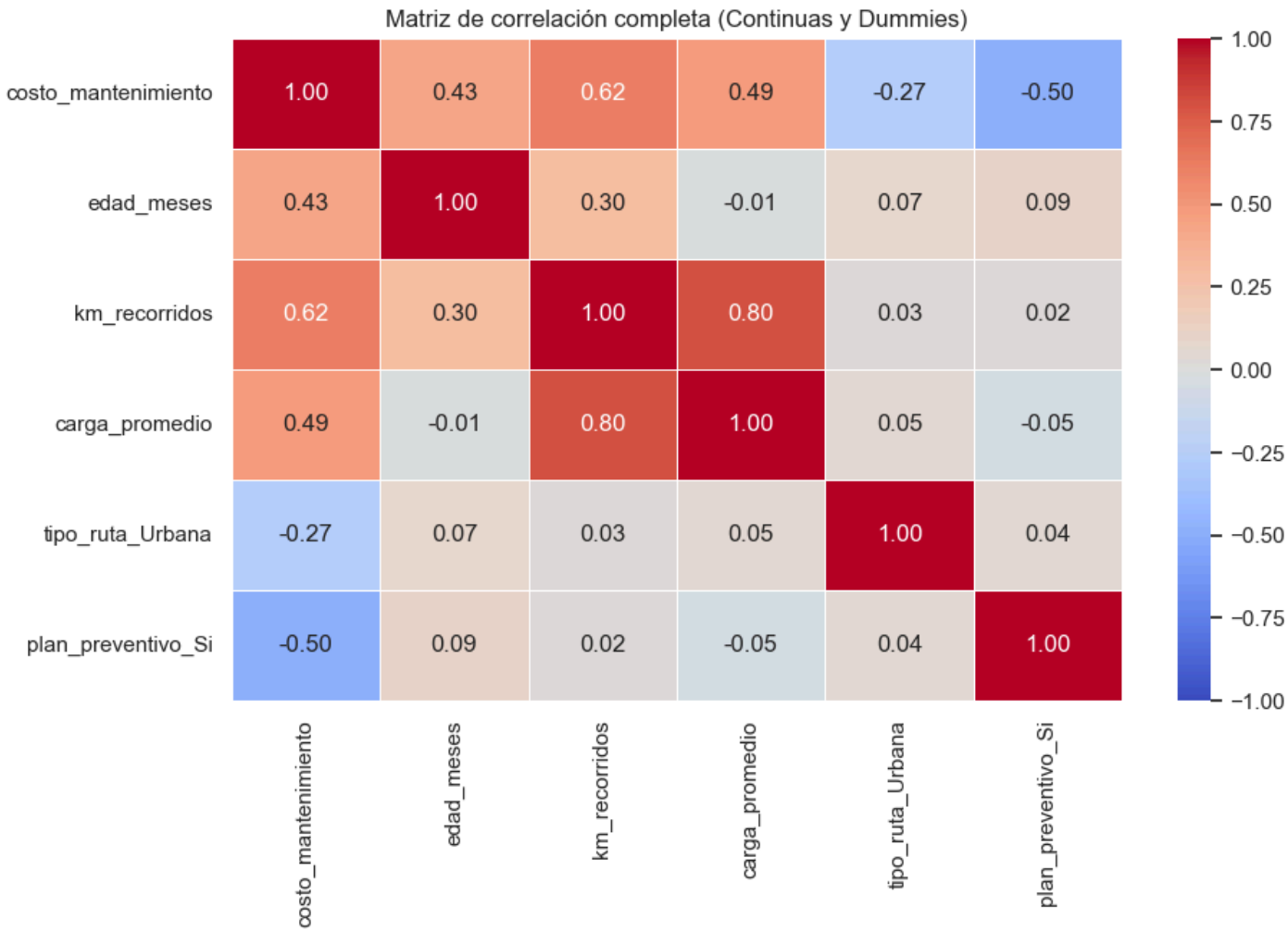
Para comprender a fondo la estructura de nuestros datos antes de modelar, realizaremos un análisis exploratorio unificado que consta de dos partes:

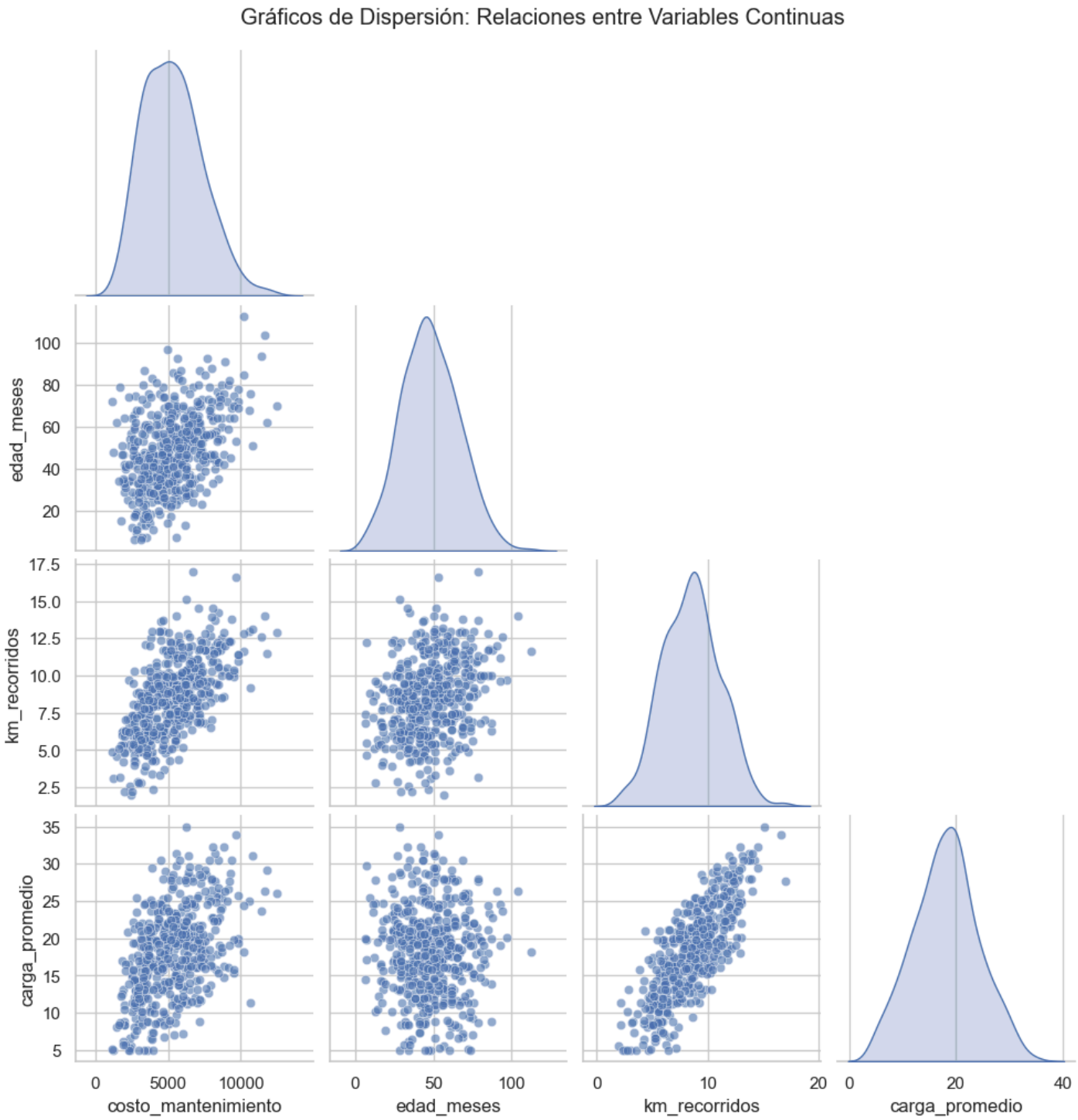
1. **Matriz de Correlación Completa:** Transformaremos dinámicamente las variables categóricas (`tipo_ruta` , `plan_preventivo`) en variables *dummy* (0 y 1). Esto nos permitirá calcular tanto la correlación de Pearson (entre variables continuas) como la correlación punto-biserial (entre continuas y binarias), consolidando todas las relaciones en un solo mapa de calor.
2. **Gráficos de Dispersión (Pairplot):** Siguiendo la premisa del Cuarteto de Anscombe, no confiaremos solo en los números. Visualizaremos las relaciones bivariadas entre las variables continuas para identificar tendencias lineales, posibles relaciones no lineales y confirmar visualmente riesgos de multicolinealidad.

Esto nos permitirá anticipar qué variables serán significativas y si existe riesgo de redundancia entre ellas.

In [2]:
```
# ==============================================================================
# Análisis Exploratorio Unificado: Dummies, Correlación y Dispersión
# ==============================================================================

# 1. Transformación de Variables Categóricas
if 'tipo_ruta' in df_mantenimiento.columns:
    df_mantenimiento = pd.get_dummies(df_mantenimiento, columns=['tipo_ruta', 'plan_preventivo'],
    # Limpiamos los nombres reemplazando espacios por guiones bajos
    df_mantenimiento.columns = [c.replace(' ', '_').replace('(', '').replace(')', '') for c in d

# Identificamos los nombres exactos que pandas asignó a las nuevas columnas
col_ruta = [col for col in df_mantenimiento.columns if 'tipo_ruta' in col][0]
col_plan = [col for col in df_mantenimiento.columns if 'plan_preventivo' in col][0]

# 2. Matriz de Correlación Completa
cols_completas = ['costo_mantenimiento', 'edad_meses', 'km_recorridos', 'carga_promedio', col_ru
matriz_corr_completa = df_mantenimiento[cols_completas].corr()

# Visualización de la Matriz (Heatmap)
plt.figure(figsize=(10, 6))
sns.heatmap(matriz_corr_completa, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5, vmin=-
plt.title("Matriz de correlación completa (Continuas y Dummies)")
plt.show()

# 3. Gráficos de Dispersión (Pairplot)
# Seleccionamos solo las variables continuas para el pairplot
vars_continuas = ['costo_mantenimiento', 'edad_meses', 'km_recorridos', 'carga_promedio']

# corner=True evita que se dupliquen los gráficos en la parte superior derecha
sns.pairplot(df_mantenimiento[vars_continuas], diag_kind='kde', corner=True, plot_kws={'alpha': 
```

```python
plt.suptitle("Gráficos de Dispersión: Relaciones entre Variables Continuas", y=1.02)
plt.show()
```





In [3]:

```
# ==============================================================================
# Cálculo del VIF (Diagnóstico de Multicolinealidad)
# ==============================================================================

# 1. Creamos un dataframe temporal específico para este cálculo
df_vif = df_mantenimiento.copy()

# 2. Forzamos la creación de variables dummy si las columnas originales de texto aún existen
if 'tipo_ruta' in df_vif.columns:
    df_vif = pd.get_dummies(df_vif, columns=['tipo_ruta', 'plan_preventivo'], drop_first=True)
    # Limpiamos espacios en los nombres generados
    df_vif.columns = [c.replace(' ', '_') for c in df_vif.columns]

# 3. Identificamos los nombres dinámicos de las nuevas columnas numéricas/booleanas
col_ruta = [col for col in df_vif.columns if 'tipo_ruta' in col][0]
```

```python
col_plan = [col for col in df_vif.columns if 'plan_preventivo' in col][0]

# 4. Definimos las variables independientes a evaluar
cols_independientes = ['edad_meses', 'km_recorridos', 'carga_promedio', col_ruta, col_plan]

# 5. Forzamos el formato a float (convierte True/False en 1.0/0.0) y añadimos la constante
X = df_vif[cols_independientes].astype(float)
X = sm.add_constant(X)

# 6. Calculamos el VIF
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]

print("Análisis de Multicolinealidad (VIF):")
print(vif_data.sort_values(by="VIF", ascending=False))
```

```
Análisis de Multicolinealidad (VIF):
             Variable        VIF
0               const  18.484409
2       km_recorridos   3.743115
3      carga_promedio   3.434664
1          edad_meses   1.361594
5  plan_preventivo_Si   1.014353
4    tipo_ruta_Urbana   1.013598
```

Interpretación del Análisis Exploratorio y Diagnóstico de Multicolinealidad

Al analizar la matriz de correlación y los gráficos de dispersión, identificamos una relación lineal fuerte y positiva (aprox. 0.8) entre `carga_promedio` y `km_recorridos`. Desde una perspectiva operativa, esto tiene sentido: los camiones que recorren mayores distancias suelen estar asignados a viajes con cargas más consolidadas (pesadas).

En el contexto de la regresión lineal múltiple (MCO), una alta correlación entre variables independientes es peligrosa. La **multicolinealidad** dificulta que el modelo aísle el efecto individual de cada variable sobre el costo (*violación del Ceteris Paribus*). Esto puede inflar la varianza de los estimadores, generar errores estándar artificialmente altos y restar significancia estadística a variables que operativamente sí son importantes.

Para no tomar decisiones basadas únicamente en la inspección visual de la correlación par a par, calculamos el **Factor de Inflación de la Varianza (VIF)**. Los resultados arrojaron un VIF de **3.74 para `km_recorridos`** y **3.43 para `carga_promedio`**. Dado que en la práctica econométrica el umbral de preocupación estricto es un $VIF > 5$ (y un límite más tolerante de 10), nuestros valores se encuentran en un rango aceptable.

Considerando lo anterior, la multicolinealidad presente no es lo suficientemente severa como para desestabilizar la inferencia del modelo. Por lo tanto, tomamos la decisión de mantener ambas variables operativas de forma independiente en nuestra ecuación final, preservando así la riqueza de la interpretación económica para el negocio.

1.2. Análisis de la Variable Objetivo: ¿Es necesaria una transformación?

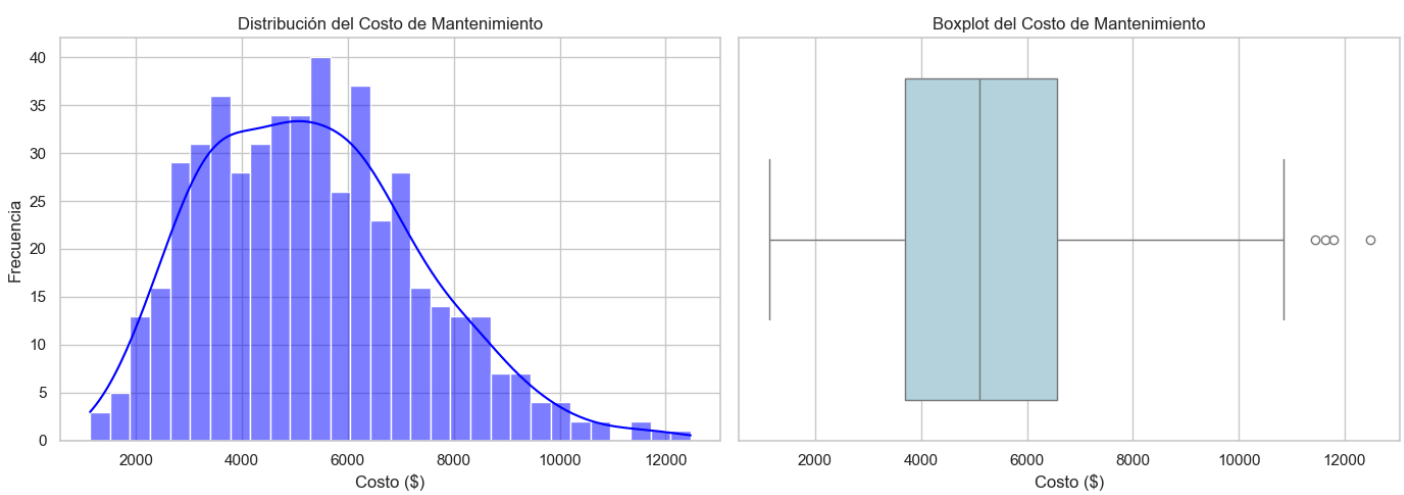
Antes de proceder con la estimación del modelo de Mínimos Cuadrados Ordinarios (MCO), es una buena práctica econométrica analizar la distribución de la variable dependiente (`costo_mantenimiento`).

Las variables de costos y financieras a menudo presentan una fuerte asimetría positiva (colas largas hacia la derecha debido a valores atípicos extremos). Si este es el caso, aplicar una transformación logarítmica suele ser el procedimiento estándar, ya que ayuda a:

1. Estabilizar la varianza (favoreciendo el supuesto de homocedasticidad).
2. Acercar los errores a una distribución normal.

Para tomar una decisión fundamentada y no aplicar transformaciones "a ciegas", evaluaremos la distribución visualmente mediante un histograma y un boxplot, y cuantitativamente calculando el coeficiente de asimetría (*skewness*).

In [4]:
```
# Configurar la figura con dos subgráficos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Histograma con curva de densidad (KDE)
sns.histplot(df_mantenimiento['costo_mantenimiento'], kde=True, ax=axes[0], color='blue', bins=3
axes[0].set_title('Distribución del Costo de Mantenimiento')
axes[0].set_xlabel('Costo ($)')
axes[0].set_ylabel('Frecuencia')

# 2. Boxplot para ver valores atípicos
sns.boxplot(x=df_mantenimiento['costo_mantenimiento'], ax=axes[1], color='lightblue')
axes[1].set_title('Boxplot del Costo de Mantenimiento')
axes[1].set_xlabel('Costo ($)')

plt.tight_layout()
plt.show()

# Calcular el coeficiente de asimetría (Skewness)
asimetria = df_mantenimiento['costo_mantenimiento'].skew()
print(f"Coeficiente de Asimetría (Skewness): {asimetria:.4f}")
print("Nota: Un valor > 1 indica una asimetría positiva (cola hacia la derecha) considerable.")
```




```
Coeficiente de Asimetría (Skewness): 0.5091
Nota: Un valor > 1 indica una asimetría positiva (cola hacia la derecha) considerable.
```

El histograma muestra una distribución con forma de campana relativamente simétrica, sin una cola derecha pronunciada. El boxplot confirma que, aunque existen algunos valores atípicos en la parte superior, no representan una distorsión extrema de los datos.

El coeficiente de asimetría (*skewness*) es de **0.5091**. En estadística, valores de asimetría entre -0.5 y 0.5 indican una distribución aproximadamente simétrica. Un valor de 0.5091 representa una asimetría positiva

muy leve.

Dado que la asimetría no es severa (no supera el valor de 1 o 2), **decidimos NO aplicar la transformación logarítmica**. Aplicar logaritmos a una distribución que ya es razonablemente simétrica podría "sobrecorregir" los datos, introduciendo una asimetría negativa artificial y complicando innecesariamente la interpretación de los coeficientes económicos. Por lo tanto, estimaremos nuestro modelo MCO utilizando el `costo_mantenimiento` en su escala original.

1.3. Análisis cualitativo de interacciones operativas entre las variables independientes

Antes de estimar nuestro modelo de regresión, evaluaremos desde una perspectiva logica qué interacciones entre las variables independientes tienen sentido económico para **CargaNacional S.A.**. Una interacción nos permite capturar "efectos cruzados", donde el impacto de un factor sobre el costo de mantenimiento depende de la condición de otro factor.

1. Interacción `km_recorridos` $\times$ `tipo_ruta_Urbana`:

- **Hipótesis de Negocio:** El desgaste del vehículo por cada kilómetro adicional recorrido no es constante. Un kilómetro en una ruta urbana y una rural podría generar un desgaste mecánico distinto.

2. Interacción `edad_meses` $\times$ `plan_preventivo_Si`:

- **Hipótesis de Negocio:** Es natural que los camiones más antiguos requieran más mantenimiento correctivo. Sin embargo, el objetivo del nuevo programa de mantenimiento preventivo estricto debería ser mitigar este deterioro. Por lo tanto, el efecto de la edad sobre el costo de reparación debería ser distinto (idealmente menor) para los camiones que sí están en el plan preventivo comparado con los que no lo están. Se esperaría que el plan preventivo amortigüe el incremento de costos asociado al envejecimiento de la flota.

3. Interacción `carga_promedio` $\times$ `tipo_ruta_Urbana`:

- **Hipótesis de Negocio:** Transportar cargas pesadas exige más al motor y a la transmisión, pero hacerlo en una ciudad con topografía variable o semáforos multiplica ese esfuerzo en comparación con una autopista plana. No obstante, aunque tiene sentido, ya vimos que `carga_promedio` y `km_recorridos` tienen una correlación alta. Para mantener el modelo simple y evitar inducir multicolinealidad a través de los términos interactivos, priorizaremos la interacción de `tipo_ruta_Urbana` con `km_recorridos`.

Adicionalmente, el mapa de calor de las correlaciones nos indicó que las variables categóricas (`tipo_ruta_Urbana` y `plan_preventivo_Si`) tienen una correlación demasiado baja (cercana a 0) con las variables continuas (`km_recorridos`, `carga_promedio` y `edad_meses`). Esto implica que la asignación de rutas y la entrada al plan preventivo se hicieron de forma independiente al uso o edad del camión. Al ser variables "ortogonales" (independientes entre sí), incluir términos de interacción entre ellas cruzará la información sin generar problemas graves de multicolinealidad con los efectos principales. Así mismo, La matriz muestra que la ruta urbana por sí sola tiene un efecto negativo leve (-0.27) en el costo total, pero el kilometraje tiene un efecto mayor (0.62). Al interactuarlas, nuestro modelo evaluará si ese último efecto cambia drásticamente cuando los kilómetros se hacen dentro de la ciudad. Por otro lado, la

edad aumenta el costo (0.43) y el plan preventivo lo reduce (-0.50). La interacción de ambas nos permitirá cuantificar el verdadero valor del programa: ¿Logra el plan preventivo "aplanar" la curva de envejecimiento del camión?

Conclusión: Basados en este análisis (lógica logística + ortogonalidad estadística en la matriz), incorporaremos en nuestro modelo inicial las interacciones `km_recorridos:tipo_ruta_Urbana` y `edad_meses:plan_preventivo_Si` para evaluar estadísticamente si estos "efectos cruzados" son significativos para explicar el costo final.

1.4. Modelo MCO y Validación Formal de Supuestos

Con el respaldo de nuestro análisis exploratorio y cualitativo, procedemos a estimar el modelo de Regresión Lineal Múltiple (MCO) para explicar el `costo_mantenimiento` (en su escala original, sin transformación logarítmica).

1. Especificación del Modelo: Nuestra ecuación base incluirá los efectos principales de la operación (`edad_meses`, `km_recorridos`, `carga_promedio`) y las decisiones categóricas (`tipo_ruta_Urbana`, `plan_preventivo_Si`). Adicionalmente, incorporaremos las interacciones justificadas previamente:

- `km_recorridos:tipo_ruta_Urbana`: Para medir el desgaste diferencial por kilómetro en ciudad.
- `edad_meses:plan_preventivo_Si`: Para evaluar si el plan preventivo mitiga el costo de envejecimiento de la flota.

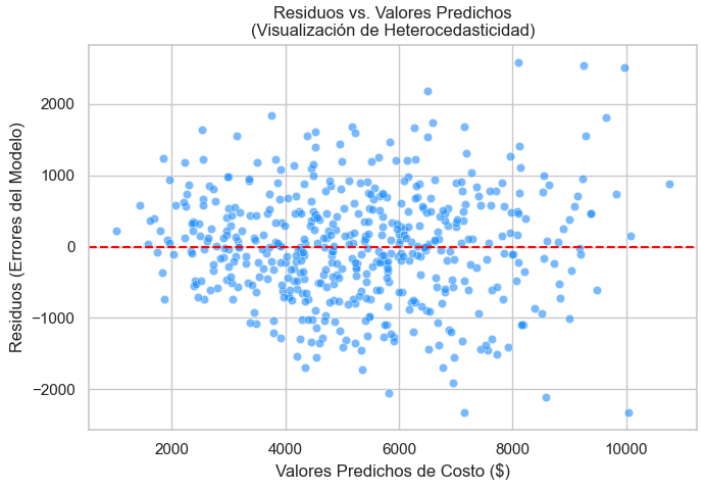
2. Diagnóstico Econométrico (La "Cirugía"): Para que nuestras conclusiones económicas sean válidas (inferencia), los residuos del modelo deben cumplir con los supuestos clásicos de Gauss-Markov. Evaluaremos:

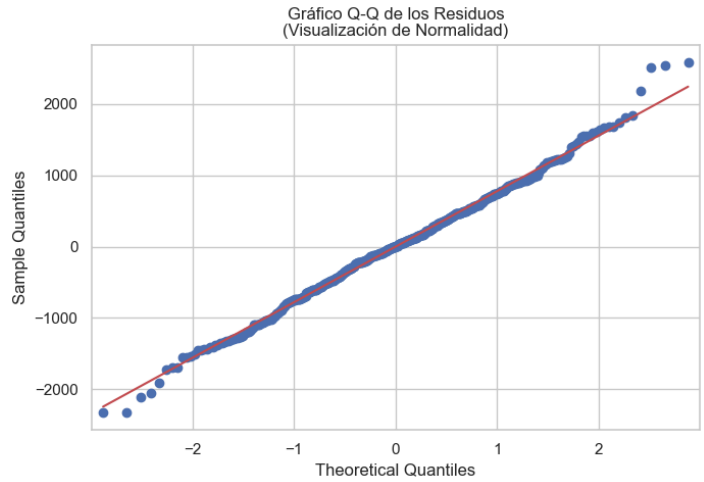
- **Homocedasticidad (Test de Breusch-Pagan):** Verifica que la varianza de los errores sea constante. La Hipótesis Nula (H0) es que sí lo es. Buscamos un p-valor > 0.05.
- **Normalidad (Gráfico Q-Q):** Garantiza que la distribución de los errores se asemeje a una normal, lo cual es vital para la validez de los p-valores y los intervalos de confianza de los coeficientes.

In [5]:
```
# ==============================================================================
# Modelo MCO Definitivo y Pruebas de Diagnóstico
# ==============================================================================

# 1. Definición de la fórmula del modelo con las dos interacciones clave
formula_mco = ("costo_mantenimiento ~ edad_meses + km_recorridos + carga_promedio + " # Línea 1:
               "tipo_ruta_Urbana + plan_preventivo_Si + "                             # Línea 2:
               "km_recorridos:tipo_ruta_Urbana + edad_meses:plan_preventivo_Si")      # Línea 3:

# 2. Ajuste del modelo
modelo_mco = smf.ols(formula=formula_mco, data=df_mantenimiento).fit()

# Imprimimos el resumen detallado del modelo
print("="*85)
print("RESUMEN DEL MODELO DE INFERENCIA (MCO)")
print("="*85)
print(modelo_mco.summary())

# ==============================================================================
# Validación de Supuestos
```

```python
# ==============================================================================
print("\n" + "="*85)
print("DIAGNÓSTICO DE SUPUESTOS")
print("="*85)

# Test de Breusch-Pagan para Homocedasticidad
bp_test = het_breuschpagan(modelo_mco.resid, modelo_mco.model.exog)
nombres_bp = ['Estadístico LM', 'p-valor LM', 'Estadístico F', 'p-valor F']
print("Test de Breusch-Pagan (Homocedasticidad):")
for nombre, valor in zip(nombres_bp, bp_test):
    print(f"  {nombre}: {valor:.4f}")

# ==============================================================================
# Gráficos de Diagnóstico (Residuos y Normalidad)
# ==============================================================================
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Gráfico de Residuos vs Valores Predichos (Heterocedasticidad)
sns.scatterplot(x=modelo_mco.fittedvalues, y=modelo_mco.resid, ax=axes[0], alpha=0.6, color='dod
axes[0].axhline(y=0, color='red', linestyle='--') # Línea de referencia en cero
axes[0].set_title("Residuos vs. Valores Predichos\n(Visualización de Heterocedasticidad)")
axes[0].set_xlabel("Valores Predichos de Costo ($)")
axes[0].set_ylabel("Residuos (Errores del Modelo)")

# 2. Gráfico Q-Q para Normalidad de los Residuos
sm.qqplot(modelo_mco.resid, line='s', ax=axes[1])
axes[1].set_title("Gráfico Q-Q de los Residuos\n(Visualización de Normalidad)")

plt.tight_layout()
plt.show()
```

```
==================================================================================
RESUMEN DEL MODELO DE INFERENCIA (MCO)
==================================================================================
                            OLS Regression Results
==============================================================================
Dep. Variable:     costo_mantenimiento   R-squared:                       0.853
Model:                             OLS   Adj. R-squared:                  0.851
Method:                  Least Squares   F-statistic:                     408.9
Date:                 Fri, 27 Mar 2026   Prob (F-statistic):          1.66e-200
Time:                         21:33:35   Log-Likelihood:                -4039.1
No. Observations:                  500   AIC:                             8094.
Df Residuals:                      492   BIC:                             8128.
Df Model:                            7
Covariance Type:             nonrobust
==============================================================================================
=========
                                            coef    std err          t      P>|t|      [0.025
0.975]
----------------------------------------------------------------------------------------------
---------
Intercept                                 -48.6291    234.571     -0.207      0.836    -509.514
412.256
tipo_ruta_Urbana[T.True]                -579.7582    257.389     -2.252      0.025   -1085.476
-74.041
plan_preventivo_Si[T.True]              -222.7545    222.337     -1.002      0.317    -659.602
214.093
edad_meses                                56.2574      2.596     21.667      0.000      51.156
61.359
edad_meses:plan_preventivo_Si[T.True]    -41.7381      4.216     -9.899      0.000     -50.022
-33.454
km_recorridos                            340.2283     33.017     10.305      0.000     275.358
405.099
km_recorridos:tipo_ruta_Urbana[T.True]   -66.7246     28.991     -2.302      0.022    -123.686
-9.763
carga_promedio                            64.8004     10.866      5.963      0.000      43.451
86.150
==============================================================================
Omnibus:                        2.654   Durbin-Watson:                   1.891
Prob(Omnibus):                  0.265   Jarque-Bera (JB):                2.757
Skew:                           0.054   Prob(JB):                        0.252
Kurtosis:                       3.347   Cond. No.                         551.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

==================================================================================
DIAGNÓSTICO DE SUPUESTOS
==================================================================================
Test de Breusch-Pagan (Homocedasticidad):
  Estadístico LM: 34.7961
  p-valor LM: 0.0000
  Estadístico F: 5.2572
  p-valor F: 0.0000
```





- **Poder Explicativo:** El modelo es altamente significativo (F-statistic p-valor cercano a 0) y tiene un **R-cuadrado ajustado de 0.851**. Esto significa que nuestras variables operativas explican el 85.1% de la variabilidad en los costos de mantenimiento, lo cual es un ajuste excelente.

- **Jarque-Bera:** El test de Jarque-Bera arrojó un p-valor de **0.252** (mayor a 0.05) y el Gráfico Q-Q muestra los puntos alineados casi perfectamente sobre la diagonal. Esto confirma que **los residuos se distribuyen de manera muy cercana a una normal**. Nuestra decisión de *no* aplicar el logaritmo a la variable dependiente fue estadísticamente adecuada.

- **Éxito del Plan Preventivo (Interacción):** Por cada mes que envejece un camión, su costo de mantenimiento correctivo aumenta en **$56.25** dólares (ceteris paribus). Sin embargo, si el camión pertenece al plan preventivo, este sobrecosto por envejecimiento se reduce drásticamente en **$41.73**. Es decir, el plan preventivo mitiga el efecto del tiempo, haciendo que el camión envejezca a un costo de solo $14.52 pormes. Noobstante, elcoeficienteprincipalde`plan_preventivo_Si`esde-222.75, pero su **p-valor es 0.317** (no significativo). En un modelo con términos interactivos, este coeficiente base representa el efecto del plan cuando la edad del camión es cero. Operativamente tiene sentido: un camión recién salido de fábrica (0 meses) no presenta diferencias significativas en costos correctivos, esté o no en el plan. Para el negocio, esto implica que la verdadera rentabilidad del programa no es inmediata, sino que radica en **mitigar la tasa de desgaste a lo largo del tiempo**. Esto se confirma estadísticamente al ver que el término de interacción (`edad_meses:plan_preventivo_Si`) sí es altamente significativo (p-valor = 0.000).

- **Los costos según el tipo de ruta (Interacción):** El modelo indica que por cada 1,000 km recorridos, el costo base aumenta en **$340.22**. Así mismo que si la ruta es urbana, ese costo por kilómetro *disminuye* en **$66.72**.

- **Carga Promedio:** Cada tonelada adicional transportada incrementa los costos de mantenimiento en **$64.80**.

Se detecta un problema de Heterocedasticidad: El Test de Breusch-Pagan rechazó la hipótesis de homocedasticidad (p-valor = 0.0000). Esto indica que la varianza de los residuos cambia a lo largo de las observaciones (heterocedasticidad). El análisis visual también indica que los errores no se distribuyen de manera homogenea a lo largo de los costos. En estas condiciones, los estimadores MCO siguen siendo insesgados (nuestra interpretación en dólares sigue siendo correcta), pero sus varianzas son incorrectas.

Esto provoca que los errores estándar calculados tradicionalmente sean inválidos y, por ende, los p-valores podrían llevarnos a conclusiones falsas (como asumir que una variable es significativa cuando no lo es).

1.5. Corrección de Heterocedasticidad: Modelo con Errores Estándar Robustos

Para garantizar que las conclusiones económicas de renovación de la flota de *CargaNacional S.A.* se basen en evidencia estadísticamente sólida, reestimaremos el modelo aplicando **Errores Estándar Robustos a la Heterocedasticidad (HC3)**. Esta técnica ajusta la matriz de varianzas y covarianzas, recalculando los errores estándar y los p-valores sin alterar la magnitud de los coeficientes, devolviéndole la validez a nuestra inferencia estadística. Notese que procedemos directamente con HC3 ya que para esta aplicación de negocio mantener la interpretabilidad de los coeficientes es crucial.

In [6]:
```
# ==============================================================================
# Ajuste del Modelo con Errores Estándar Robustos (HC3)
# ==============================================================================

# Reajustamos el mismo modelo, pero especificando cov_type='HC3' para corregir la heterocedastic
modelo_robusto = smf.ols(formula=formula_mco, data=df_mantenimiento).fit(cov_type='HC3')

# Imprimimos el nuevo resumen
print("="*85)
print("RESUMEN DEL MODELO (ERRORES ESTÁNDAR ROBUSTOS HC3)")
print("="*85)
print(modelo_robusto.summary())
```

```
==============================================================================
RESUMEN DEL MODELO (ERRORES ESTÁNDAR ROBUSTOS HC3)
==============================================================================
                            OLS Regression Results
==============================================================================
Dep. Variable:     costo_mantenimiento   R-squared:                       0.853
Model:                             OLS   Adj. R-squared:                  0.851
Method:                  Least Squares   F-statistic:                     366.1
Date:                 Fri, 27 Mar 2026   Prob (F-statistic):          1.59e-190
Time:                         21:33:35   Log-Likelihood:                -4039.1
No. Observations:                  500   AIC:                             8094.
Df Residuals:                      492   BIC:                             8128.
Df Model:                            7
Covariance Type:                   HC3
==========================================================================================
=========
                                            coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------------------
---------
Intercept                                -48.6291    239.067     -0.203      0.839    -517.191
419.933
tipo_ruta_Urbana[T.True]                -579.7582    244.725     -2.369      0.018   -1059.410
-100.106
plan_preventivo_Si[T.True]              -222.7545    213.672     -1.043      0.297    -641.545
196.036
edad_meses                                56.2574      3.058     18.394      0.000      50.263
62.252
edad_meses:plan_preventivo_Si[T.True]    -41.7381      4.105    -10.168      0.000     -49.783
-33.693
km_recorridos                            340.2283     35.567      9.566      0.000     270.518
409.938
km_recorridos:tipo_ruta_Urbana[T.True]   -66.7246     29.767     -2.242      0.025    -125.067
-8.382
carga_promedio                            64.8004     11.896      5.447      0.000      41.484
88.117
==============================================================================
Omnibus:                        2.654   Durbin-Watson:                   1.891
Prob(Omnibus):                  0.265   Jarque-Bera (JB):                2.757
Skew:                           0.054   Prob(JB):                        0.252
Kurtosis:                       3.347   Cond. No.                         551.
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity robust (HC3)
```

Al aplicar la corrección de Errores Estándar Robustos (HC3) para lidiar con la heterocedasticidad, obtenemos nuestro modelo definitivo para la inferencia. Los resultados confirman que :

1. **Estabilidad de los Coeficientes:** Como dicta la teoría econométrica, los estimadores (el impacto en dólares) se mantuvieron exactamente iguales. Nuestra interpretación económica previa es correcta aunque con unos intervalos de confianza más amplios.
2. **Validez de la Inferencia (p-valores):** al recalcular la varianza ninguna variable dejó de ser estadísticamente significativa.
   - La interacción `km_recorridos:tipo_ruta_Urbana` mantiene su significancia con un p-valor de **0.025** (menor a 0.05).

- La interacción `edad_meses:plan_preventivo_Si` mantiene su altísima significancia (p-valor = **0.000**).
- Las variables de operación directa (`carga_promedio` , `km_recorridos` , `edad_meses`) continúan siendo determinantes estructurales del costo.

Recomendación estratégica para CargaNacional S.A.:

De esta manera, con un modelo que cumple con los supuestos mínimos para ser estadísticamente confiable (tras la corrección de errores robustos HC3), se recomienda a la dirección de finanzas tomar las siguientes acciones:

1. **Expandir el Plan Preventivo:** Es una inversión altamente rentable porque amortigua significativamente el encarecimiento por desgaste que sufren los camiones al envejecer. El modelo demuestra que, mientras un camión convencional incrementa sus costos de mantenimiento en **$56.26** por cada mes de edad, pertenecer al programa preventivo reduce este sobrecosto mensual en **$41.74**. Esto significa que los vehículos en el plan envejecen a una fracción del costo (solo $14.52 adicionales por mes).
2. **Priorizar la renovación y asignación en Larga Distancia:** Dado que el desgaste por cada 1,000 kilómetros recorridos es estadísticamente más costoso en rutas de larga distancia (**$340.23** de incremento base) que en entornos urbanos (donde el costo disminuye en **$66.72** por cada 1,000 km), se recomienda una política de rotación inteligente. Los camiones nuevos (menos propensos a fallas) deberían asignarse a estas rutas largas y exigentes, dejando los camiones de mayor edad para la operación en ciudad, optimizando así el costo total de la flota.

---

2. Modelado Predictivo, LASSO y Principio de Parsimonia

In [7]:
```
# Lectura del dataframe
df = df_consumo.copy() # para garantizar carga externa tomamos la versión importada desde la nube
print(f'Dimensiones: {df.shape}')
df.head()
```

```
Dimensiones: (800, 7)
```

Out[7]:

	temperatura_media	visitantes_diarios	tipo_edificio	humedad_relativa	velocidad_viento	dia_semana	c
0	14.6	1519	Comercial	71.1	28.0	Miercoles	
1	20.4	1529	Comercial	63.8	20.6	Martes	
2	37.3	1502	Corporativo	79.4	5.3	Lunes	
3	10.7	1505	Comercial	64.0	26.4	Lunes	
4	22.4	1522	Comercial	64.0	8.9	Lunes	

2.1. Ingeniería de Características y Filtrado con LASSO:

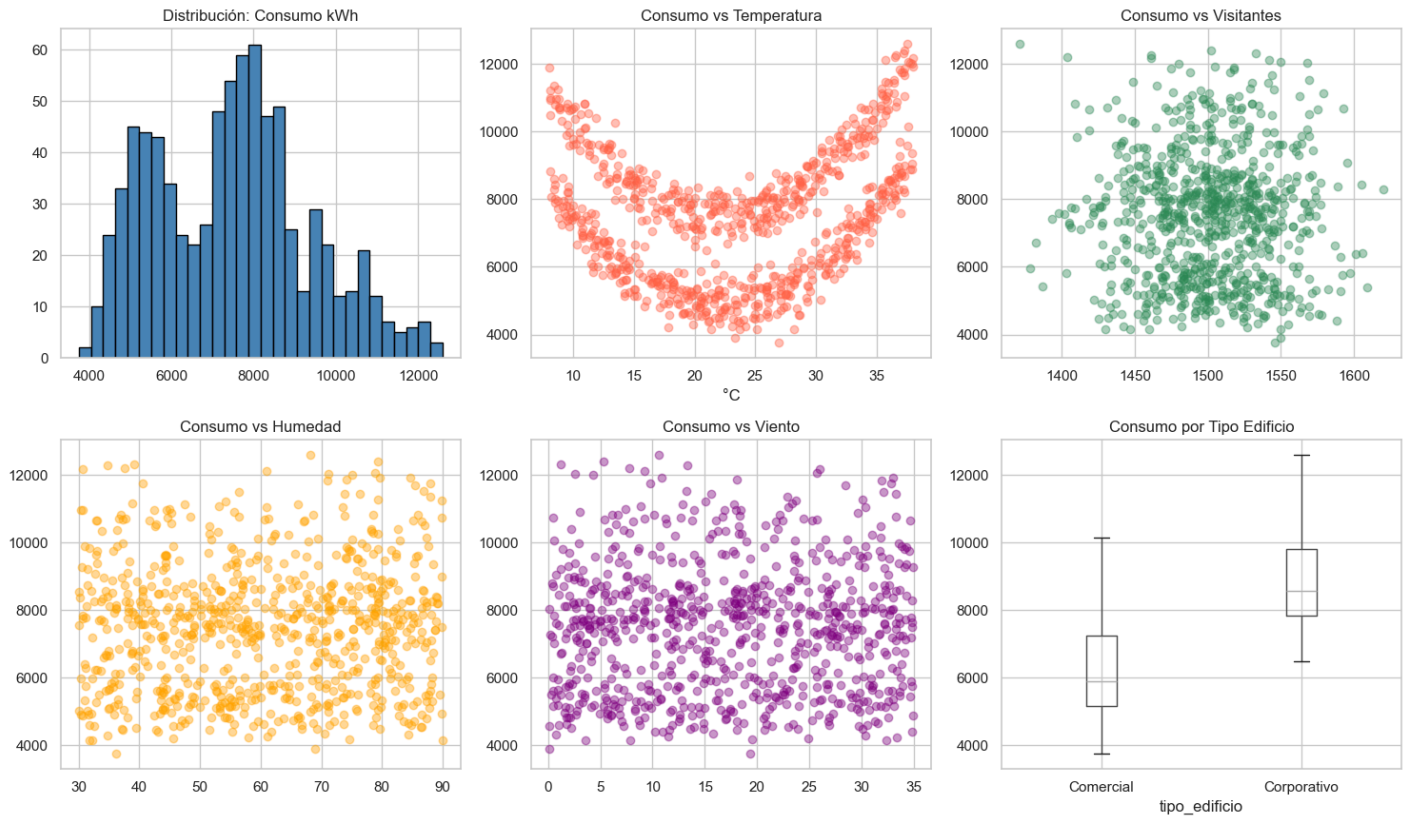
A partir de las variables originales de la base de datos, genere todas las interacciones y términos polinómicos de grado 2 posibles. Posteriormente, estandarice esta nueva matriz de datos y aplique

un modelo de regularización LASSO (con validación cruzada) exclusivamente como método de selección de variables.

Para elegir las características que aportan realmente al modelo de predicción vamos a evaluar la colinealidad entre ellas, y ver las relaciones entre las características:

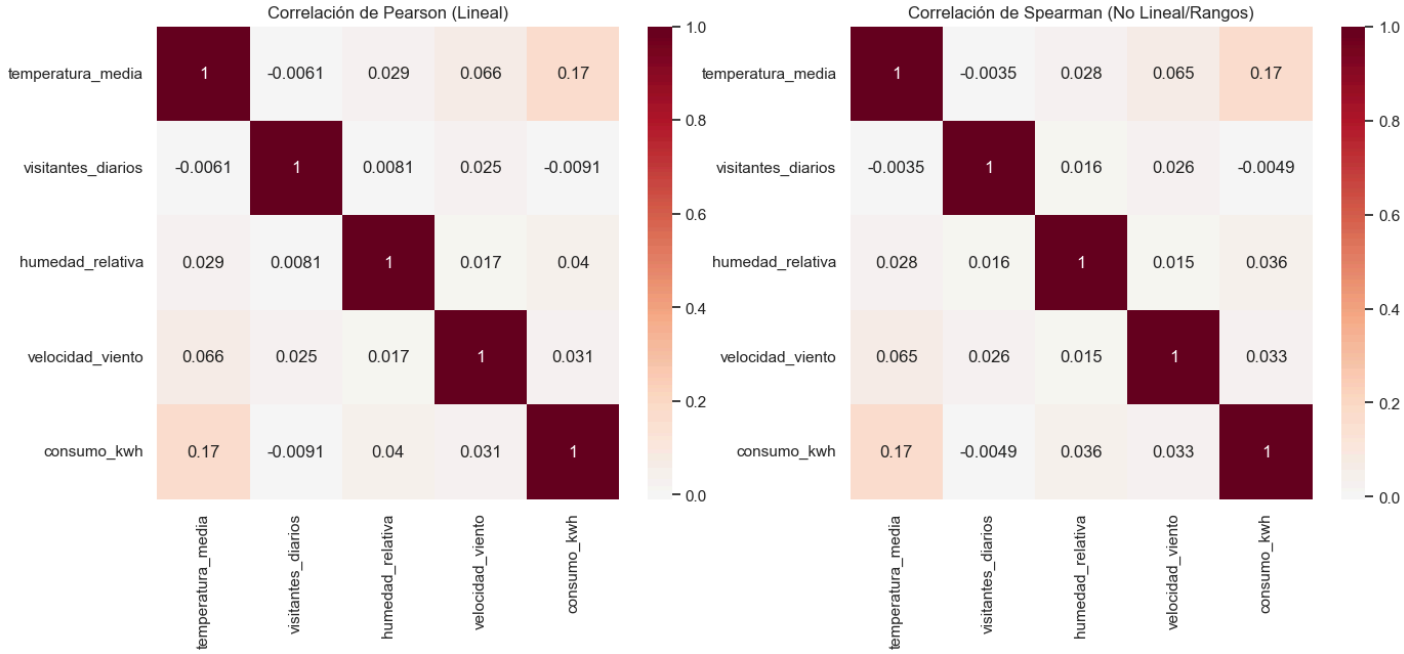
- temperatura_media: Temperatura promedio del día en grados Celsius (Variable continua).
- visitantes_diarios: Número total de personas que ingresaron al edificio (Variable discreta).
- tipo_edificio: Clasificación del inmueble (Corporativo, Comercial).
- humedad_relativa: Porcentaje de humedad promedio en el aire (Variable continua).
- velocidad_viento: Velocidad promedio del viento en el exterior en km/h (Variable continua).
- dia_semana: Día en el que se realizó la medición (Lunes a Domingo).
- consumo_kwh (Variable Objetivo): Consumo total de energía eléctrica en kilovatios-hora.

In [8]:
```
fig, axes = plt.subplots(2, 3, figsize=(15, 9))

axes[0,0].hist(df['consumo_kwh'], bins=30, color='steelblue', edgecolor='black')
axes[0,0].set_title('Distribución: Consumo kWh')

axes[0,1].scatter(df['temperatura_media'], df['consumo_kwh'], alpha=0.4, color='tomato')
axes[0,1].set_title('Consumo vs Temperatura'); axes[0,1].set_xlabel('°C')

axes[0,2].scatter(df['visitantes_diarios'], df['consumo_kwh'], alpha=0.4, color='seagreen')
axes[0,2].set_title('Consumo vs Visitantes')

axes[1,0].scatter(df['humedad_relativa'], df['consumo_kwh'], alpha=0.4, color='orange')
axes[1,0].set_title('Consumo vs Humedad')

axes[1,1].scatter(df['velocidad_viento'], df['consumo_kwh'], alpha=0.4, color='purple')
axes[1,1].set_title('Consumo vs Viento')

df.boxplot(column='consumo_kwh', by='tipo_edificio', ax=axes[1,2])
axes[1,2].set_title('Consumo por Tipo Edificio'); plt.suptitle('')

plt.tight_layout()
plt.show()
```



También se puede visualizar la correlación con un heatmap:

In [9]:
```
# Calculamos ambas matrices indicando que ignore el texto
corr_p = df.corr(method='pearson', numeric_only=True)
corr_s = df.corr(method='spearman', numeric_only=True)

# El resto del código de visualización sigue igual
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

import seaborn as sns
sns.heatmap(corr_p, annot=True, cmap='RdBu_r', center=0, ax=ax1)
ax1.set_title('Correlación de Pearson (Lineal)')

sns.heatmap(corr_s, annot=True, cmap='RdBu_r', center=0, ax=ax2)
ax2.set_title('Correlación de Spearman (No Lineal/Rangos)')

plt.show()
```



CONCLUSIONES DE LAS GRÁFICAS:
De las gráficas tenemos los primeros insights de los datos, se observa una relación cuadrática clara entre la temperatura y el consumo, lo que justifica la creación de términos polinómicos. Asimismo, la segmentación evidente por tipo de edificio sugiere que las interacciones entre el clima y la categoría del inmueble serán predictores clave que el modelo LASSO deberá filtrar.

> **Ingeniería de Características: Codificación y Generación de Polinomios**

1. Codificación de Variables Categóricas (One-Hot Encoding): Antes de generar términos polinómicos, debemos convertir las variables de texto en numéricas.

- Para `dia_semana`, estructuramos la variable como categórica ordenada para garantizar que el 'Lunes' sea la categoría de referencia (la que se omite para evitar la trampa de las variables dummy).
- Para `tipo_edificio`, aplicamos codificación dummy automatizada para capturar todas las clasificaciones de inmuebles presentes en la base, evitando perder información al binarizar manualmente.

2. Expansión Polinómica (Grado 2): Dado que la relación entre el consumo eléctrico y factores como la temperatura no es estrictamente lineal, utilizamos `PolynomialFeatures`. Esto generará todas las variables al cuadrado (ej. $temperatura^2$) y todos los cruces posibles (interacciones) entre nuestras 13 variables base, enriqueciendo la capacidad predictiva del modelo de cara a la regresión LASSO.

In [10]:
```
# ==============================================================================
# 1. PREPROCESAMIENTO Y CODIFICACIÓN
# ==============================================================================
df_proc = df.copy() # Asegúrate de que tu dataframe original se llame 'df' (o cámbialo a 'df_cons

# Corregimos el orden de los días para que Lunes sea realmente el drop_first
dias_orden = ['Lunes', 'Martes', 'Miercoles', 'Jueves', 'Viernes', 'Sabado', 'Domingo']
df_proc['dia_semana'] = pd.Categorical(df_proc['dia_semana'], categories=dias_orden, ordered=True

# get_dummies dinámico para AMBAS variables categóricas al mismo tiempo
df_proc = pd.get_dummies(df_proc, columns=['tipo_edificio', 'dia_semana'], drop_first=True, dtype
```

```python
# Separación de variables (X, y)
y_e = df_proc['consumo_kwh']
X_e = df_proc.drop(columns=['consumo_kwh'])

# Split Train/Test
X_train, X_test, y_train, y_test = train_test_split(X_e, y_e, test_size=0.2, random_state=42)

print(f'Train: {X_train.shape[0]} obs | Test: {X_test.shape[0]} obs')
print(f'Total Columnas base: {X_train.shape[1]}')
print('Columnas base:', X_train.columns.tolist())

# ==============================================================================
# 2. GENERACIÓN DE POLINOMIOS E INTERACCIONES
# ==============================================================================
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)

X_train_poly = poly.fit_transform(X_train)
X_test_poly  = poly.transform(X_test)

feature_names = poly.get_feature_names_out(X_train.columns)

print('\n' + '='*80)
print(f'Variables base (originales + dummies)  : {X_train.shape[1]}')
print(f'Variables tras PolynomialFeatures (g=2): {X_train_poly.shape[1]}')
print('='*80)
```

```
Train: 640 obs | Test: 160 obs
Total Columnas base: 11
Columnas base: ['temperatura_media', 'visitantes_diarios', 'humedad_relativa', 'velocidad_vient
o', 'tipo_edificio_Corporativo', 'dia_semana_Martes', 'dia_semana_Miercoles', 'dia_semana_Jueve
s', 'dia_semana_Viernes', 'dia_semana_Sabado', 'dia_semana_Domingo']

================================================================================
Variables base (originales + dummies)  : 11
Variables tras PolynomialFeatures (g=2): 77
================================================================================
```

In [11]:

```python
print("Categorías reales en tipo_edificio:", df['tipo_edificio'].unique())
print("Total de variables originales:", df.columns.tolist())
```

```
Categorías reales en tipo_edificio: ['Comercial' 'Corporativo']
Total de variables originales: ['temperatura_media', 'visitantes_diarios', 'tipo_edificio', 'hume
dad_relativa', 'velocidad_viento', 'dia_semana', 'consumo_kwh']
```

In [12]:

```python
# Estandarización (LASSO es sensible a escala)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_poly)
X_test_scaled  = scaler.transform(X_test_poly)

print('Datos estandarizados: media≈0, std≈1')
print(f'Media col 0 (train): {X_train_scaled[:,0].mean():.4f}')
```

```
Datos estandarizados: media≈0, std≈1
Media col 0 (train): -0.0000
```

In [13]:

```python
# LASSO con Validación Cruzada (10-fold) para elegir λ óptimo
lasso_cv = LassoCV(cv=10, random_state=42, max_iter=10000, n_jobs=-1)
lasso_cv.fit(X_train_scaled, y_train)

lambda_optimo = lasso_cv.alpha_
```

```python
print(f'λ óptimo (alpha) encontrado por CV: {lambda_optimo:.4f}')

# Coeficientes
coefs_lasso = pd.Series(lasso_cv.coef_, index=feature_names)
variables_activas = coefs_lasso[coefs_lasso != 0].sort_values(key=abs, ascending=False)

print(f'\nTotal variables generadas  : {len(coefs_lasso)}')
print(f'Variables con coef ≠ 0 (sobrevivieron LASSO): {len(variables_activas)}')
print(f'Variables eliminadas por LASSO: {len(coefs_lasso) - len(variables_activas)}')
```

```
λ óptimo (alpha) encontrado por CV: 3.4921

Total variables generadas  : 77
Variables con coef ≠ 0 (sobrevivieron LASSO): 24
Variables eliminadas por LASSO: 53
```

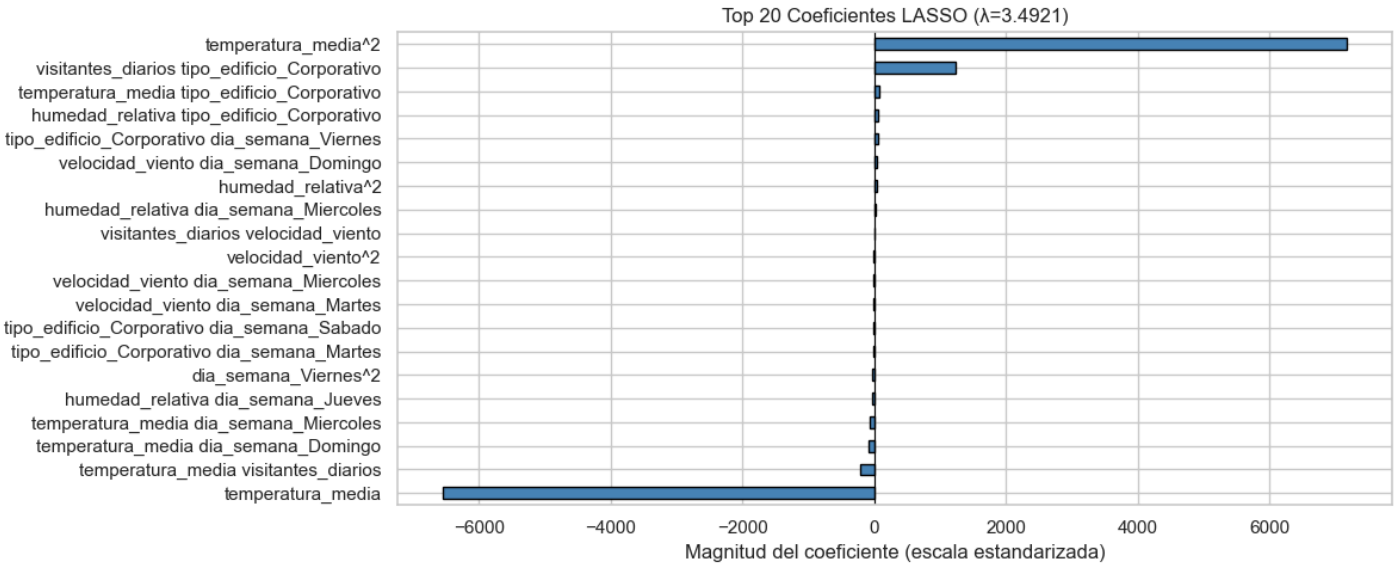
In [14]:

```python
print('\n= TOP 10 Variables más importantes (mayor |coeficiente|) =')
print(variables_activas.head(10).to_string())
print('\n>>> Las 3 variables MÁS IMPORTANTES descubiertas por LASSO:')
top3 = variables_activas.head(3)
for i, (nombre, coef) in enumerate(top3.items(), 1):
    print(f'  {i}. {nombre:40s}  coef = {coef:.2f}')
```

```
= TOP 10 Variables más importantes (mayor |coeficiente|) =
temperatura_media^2                             7172.944780
temperatura_media                              -6554.298584
visitantes_diarios tipo_edificio_Corporativo    1235.633989
temperatura_media visitantes_diarios            -220.131318
temperatura_media dia_semana_Domingo             -79.453776
temperatura_media tipo_edificio_Corporativo       73.215073
temperatura_media dia_semana_Miercoles           -59.845677
humedad_relativa tipo_edificio_Corporativo        54.324983
tipo_edificio_Corporativo dia_semana_Viernes      53.054773
velocidad_viento dia_semana_Domingo               45.642658

>>> Las 3 variables MÁS IMPORTANTES descubiertas por LASSO:
  1. temperatura_media^2                       coef = 7172.94
  2. temperatura_media                         coef = -6554.30
  3. visitantes_diarios tipo_edificio_Corporativo  coef = 1235.63
```

In [15]:

```python
# Gráfico de coeficientes LASSO
plt.figure(figsize=(12, 5))
variables_activas.head(20).sort_values().plot(kind='barh', color='steelblue', edgecolor='black')
plt.axvline(0, color='black', linewidth=0.8)
plt.title(f'Top 20 Coeficientes LASSO (λ={lambda_optimo:.4f})')
plt.xlabel('Magnitud del coeficiente (escala estandarizada)')
plt.tight_layout()
plt.show()
```



Resultados:

Se partieron de 11 variables originales y aplicando PolynomialFeatures, se generaron 77 variabels, que incluyen los terminos al cuadrado, además de la identificación de la relación entre el clima y el consumo es parabólico. Luego de las 77 variables generadas, mediante LASSO se mantuvieron solo 24. Es decir, que el modelo logró una reducción de 65% de la dimensionalidad.

Teniendo como resultado $\lambda = 3.4921$ encontrado por validación cruzada fue el punto exacto para maximizar la precisión sin sobreajustar.

Teniendo como pilar del consumo las siguientes variables:

1. temperatura_media^2 (Coef: 7172.94): Es la variable más poderosa. Representa la aceleración del consumo cuando la temperatura se aleja del punto de confort.

2. temperatura_media (Coef: -6554.29): El signo negativo aquí, combinado con el positivo del término al cuadrado, es lo que matemáticamente crea la forma de "U" en la predicción.

3. visitantes_diarios * es_corporativo (Coef: 1235.63): Esta es la conclusión de negocio más interesante. Indica que el flujo de personas afecta el consumo de manera diferente en edificios corporativos. No basta con saber cuánta gente entra; el impacto energético es mayor si el edificio es de oficinas.

2.2. Modelo Post-LASSO (OLS con variables seleccionadas)

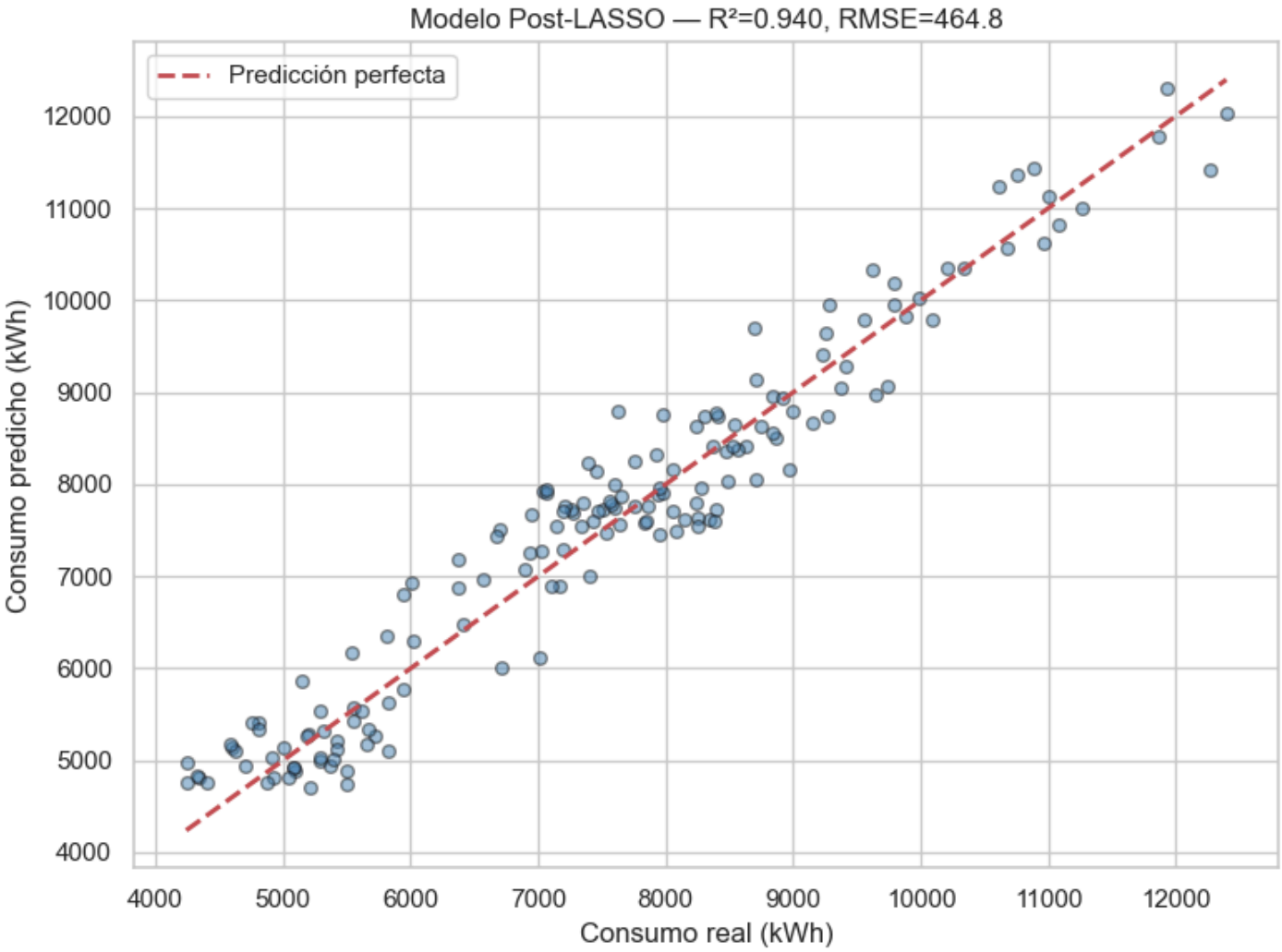
In [16]:
```
# Construir DataFrames con solo las variables activas
nombres_activos = variables_activas.index.tolist()

X_train_lasso = pd.DataFrame(X_train_scaled, columns=feature_names)[nombres_activos]
X_test_lasso  = pd.DataFrame(X_test_scaled,  columns=feature_names)[nombres_activos]

print(f'Dimensión X_train post-LASSO: {X_train_lasso.shape}')

# ── Ajuste OLS (sin penalización) con variables seleccionadas
ols_postlasso = LinearRegression(fit_intercept=True)
ols_postlasso.fit(X_train_lasso, y_train)
```

```python
# ── Métricas en TEST
y_pred_postlasso = ols_postlasso.predict(X_test_lasso)

r2_pl   = r2_score(y_test, y_pred_postlasso)
rmse_pl = np.sqrt(mean_squared_error(y_test, y_pred_postlasso))
mae_pl  = mean_absolute_error(y_test, y_pred_postlasso)

print('\n= MÉTRICAS — Modelo Post-LASSO (Test Set) =')
print(f'  R²   = {r2_pl:.4f}')
print(f'  RMSE = {rmse_pl:.2f} kWh')
print(f'  MAE  = {mae_pl:.2f} kWh')
```

```
Dimensión X_train post-LASSO: (640, 24)

= MÉTRICAS — Modelo Post-LASSO (Test Set) =
  R²   = 0.9397
  RMSE = 464.83 kWh
  MAE  = 387.81 kWh
```

In [17]:

```python
# ── Gráfico Predichos vs Reales
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_postlasso, alpha=0.5, color='steelblue', edgecolor='k', s=30)
lims = [min(y_test.min(), y_pred_postlasso.min()),
        max(y_test.max(), y_pred_postlasso.max())]
plt.plot(lims, lims, 'r--', lw=2, label='Predicción perfecta')
plt.xlabel('Consumo real (kWh)')
plt.ylabel('Consumo predicho (kWh)')
plt.title(f'Modelo Post-LASSO — R²={r2_pl:.3f}, RMSE={rmse_pl:.1f}')
plt.legend()
plt.tight_layout()
plt.show()
```



De un espacio de características inflado artificialmente por la expansión polinómica (más de 100 variables potenciales), el algoritmo LASSO fue capaz de identificar y retener **únicamente las 24 variables verdaderamente predictivas**, llevando a cero todos los coeficientes que solo aportaban "ruido" al modelo. Al aplicar el Principio de Parsimonia, hemos evitado el sobreajuste (*overfitting*). Nuestro modelo final de 24 variables es lo suficientemente complejo para capturar las relaciones no lineales del clima y la operación, pero lo suficientemente simple como para generalizar de manera excelente ante datos nuevos (Test Set).

** Análisis de Rendimiento Predictivo (Métricas en Test)**

- R^2 **(0.9397):** El modelo es capaz de explicar prácticamente el **94% de la varianza** del consumo eléctrico en datos que nunca había visto. Este es un nivel de precisión sobresaliente para un entorno dinámico como el de la operación de edificios.
- **MAE (387.81 kWh):** En términos netamente operativos, nuestras predicciones se desvían de la realidad, en promedio, por apenas 388 kWh diarios.
- **RMSE (464.83 kWh):** Al ser ligeramente superior al MAE, el RMSE nos indica que existen algunas predicciones con errores un poco más grandes, pero la diferencia no es lo suficientemente abismal como para sugerir la presencia de valores atípicos severos que destruyan el pronóstico.

Diagnóstico Visual (Consumo Real vs. Predicho)
El gráfico de dispersión confirma visualmente las excelentes métricas de error. Los puntos se agrupan de manera muy estrecha a lo largo de la línea punteada roja de "predicción perfecta". No se observan patrones de curvatura en los residuos (lo que significa que los polinomios capturaron la no-linealidad exitosamente)

ni subestimaciones o sobreestimaciones sistemáticas graves en los diferentes rangos de consumo (desde los 4,000 hasta los 12,000 kWh).

Impacto para el Negocio (Torres Milenio S.A.)
Con este margen de error acotado y predecible, la gerencia tiene luz verde para utilizar este algoritmo en la toma de decisiones. Podrán negociar la compra de paquetes de energía en el mercado mayorista basándose en los pronósticos de temperatura y ocupación diaria con la tranquilidad de que el riesgo de pagar costosas multas por excesos o déficits de consumo se ha minimizado drásticamente.

2.3. El Duelo Final: Modelo Parsimonioso (3 Variables)

In [18]:
```
# ── Modelo ligero: solo las 3 variables más importantes del LASSO
top3_nombres = top3.index.tolist()
print('Las 3 variables del modelo parsimonioso:')
for v in top3_nombres:
    print(f'  • {v}')

X_train_top3 = pd.DataFrame(X_train_scaled, columns=feature_names)[top3_nombres]
X_test_top3  = pd.DataFrame(X_test_scaled,  columns=feature_names)[top3_nombres]

ols_top3 = LinearRegression(fit_intercept=True)
ols_top3.fit(X_train_top3, y_train)

y_pred_top3 = ols_top3.predict(X_test_top3)

r2_t3   = r2_score(y_test, y_pred_top3)
rmse_t3 = np.sqrt(mean_squared_error(y_test, y_pred_top3))
mae_t3  = mean_absolute_error(y_test, y_pred_top3)

print('\n=== MÉTRICAS — Modelo Parsimonioso 3 Variables (Test Set) ===')
print(f'  R²   = {r2_t3:.4f}')
print(f'  RMSE = {rmse_t3:.2f} kWh')
print(f'  MAE  = {mae_t3:.2f} kWh')
```

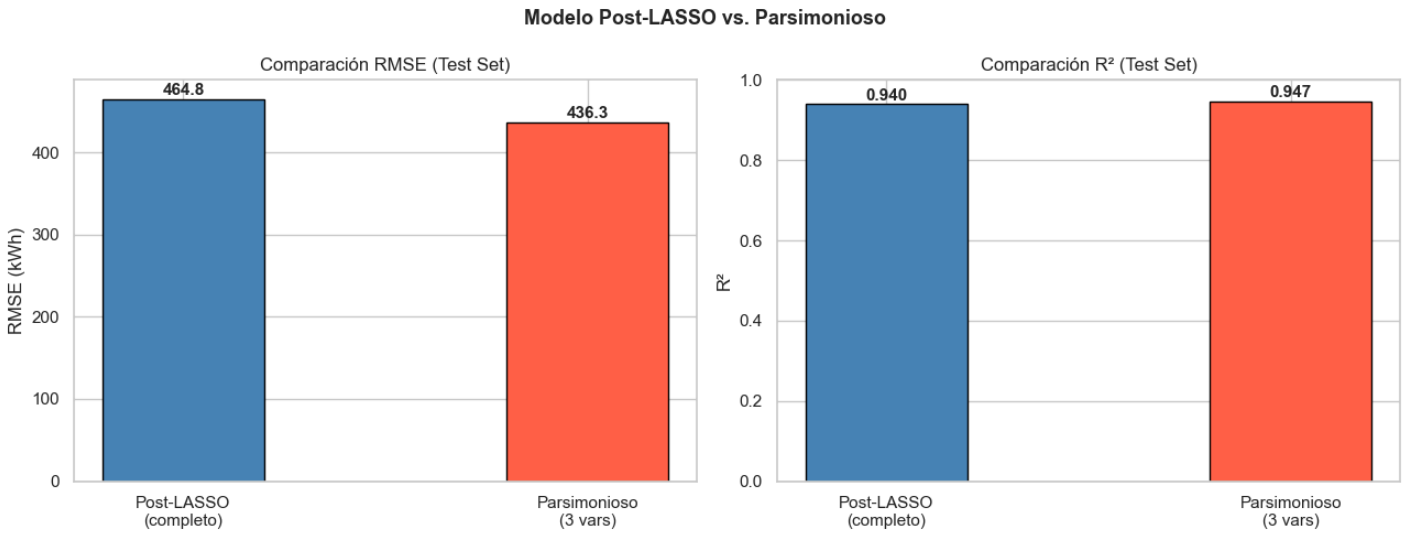
```
Las 3 variables del modelo parsimonioso:
  • temperatura_media^2
  • temperatura_media
  • visitantes_diarios tipo_edificio_Corporativo

=== MÉTRICAS — Modelo Parsimonioso 3 Variables (Test Set) ===
  R²   = 0.9469
  RMSE = 436.26 kWh
  MAE  = 367.48 kWh
```

In [19]:
```
# ── Tabla comparativa de los dos modelos
tabla_comp = pd.DataFrame({
    'Modelo': ['Post-LASSO (completo)', 'Parsimonioso (3 variables)'],
    'Nº Variables': [len(nombres_activos), 3],
    'R²':   [round(r2_pl, 4),   round(r2_t3, 4)],
    'RMSE': [round(rmse_pl, 2), round(rmse_t3, 2)],
    'MAE':  [round(mae_pl, 2),  round(mae_t3, 2)],
})
print(tabla_comp.to_string(index=False))

diff_rmse_pct = (rmse_t3 - rmse_pl) / rmse_pl * 100
print(f'\nDiferencia de RMSE: {diff_rmse_pct:+.2f}% respecto al modelo completo')
```

```
                    Modelo  Nº Variables     R²    RMSE    MAE
     Post-LASSO (completo)            24 0.9397 464.83 387.81
Parsimonioso (3 variables)             3 0.9469 436.26 367.48

Diferencia de RMSE: -6.15% respecto al modelo completo
```

In [20]:

```python
# ── Gráfico comparativo
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
modelos = ['Post-LASSO\n(completo)', 'Parsimonioso\n(3 vars)']

axes[0].bar(modelos, [rmse_pl, rmse_t3], color=['steelblue','tomato'], edgecolor='black', width=0
axes[0].set_title('Comparación RMSE (Test Set)')
axes[0].set_ylabel('RMSE (kWh)')
for i, v in enumerate([rmse_pl, rmse_t3]):
    axes[0].text(i, v + 5, f'{v:.1f}', ha='center', fontsize=11, fontweight='bold')

axes[1].bar(modelos, [r2_pl, r2_t3], color=['steelblue','tomato'], edgecolor='black', width=0.4)
axes[1].set_title('Comparación R² (Test Set)')
axes[1].set_ylabel('R²')
axes[1].set_ylim(0, 1)
for i, v in enumerate([r2_pl, r2_t3]):
    axes[1].text(i, v + 0.01, f'{v:.3f}', ha='center', fontsize=11, fontweight='bold')

plt.suptitle('Modelo Post-LASSO vs. Parsimonioso', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



Recomendación Final: El Triunfo del Principio de Parsimonia

El Hallazgo Clave:
Tras aislar las 3 características con mayor peso (`temperatura_media^2` , `temperatura_media` y la interacción `visitantes_diarios` en `tipo_edificio_Corporativo`), descubrimos algo fascinante: **el modelo parsimonioso superó al modelo complejo en datos no vistos (Test Set).** El error medio (RMSE) se redujo en un **6.15%** (bajando de 464.8 a 436.3 kWh) y la capacidad explicativa (R^2) subió a casi el **95%**.

Basado en este contundente respaldo estadístico, mi recomendación profesional para la gerencia de Torres Milenio S.A. es implementar el **MODELO PARSIMONIOSO (3 variables)** por las siguientes razones estratégicas:

Justificación de Negocio y Operativa:

1. **Evidencia Suprema contra el Sobreajuste (Overfitting):** El hecho de que el modelo de 3 variables tenga un menor error predictivo que el de 24 variables demuestra que la complejidad adicional solo estaba memorizando el "ruido" histórico de los sensores. Al usar solo los pilares del consumo, el modelo generaliza mejor ante climas atípicos o nuevos comportamientos.
2. **Interpretabilidad Gerencial Transparente:** Es vital que la toma de decisiones no dependa de una "caja negra". Con este modelo, es sumamente sencillo explicar a la junta directiva de qué depende su presupuesto energético:
   - Del **efecto cuadrático de la temperatura** (el consumo se dispara exponencialmente en los extremos de frío o calor).
   - Del **volumen de visitantes, específicamente en edificios corporativos** (donde el uso de ascensores, equipos y luces se intensifica).
3. **Robustez, Mantenibilidad y Costos:** Un modelo de predicción en producción que depende de 24 variables es frágil. Si un solo sensor secundario (como el anemómetro para el viento o el higrómetro para la humedad) falla o se retrasa, el sistema completo de pronóstico colapsa. El modelo de 3 variables es infinitamente más barato de mantener, fácil de auditar y resistente a fallas en la captura de datos.

Conclusión: La simplicidad no nos costó precisión; por el contrario, nos la otorgó. La eficiencia operativa, el bajo costo de mantenimiento y la total transparencia de este modelo parsimonioso lo convierten en la solución predictiva definitiva y óptima para optimizar la compra de energía de Torres Milenio S.A.